

System/Software Requirements Definition Process for OptArrow

1. System Context & Overview

1.1 Overview of OptArrow's Architecture

OptArrow serves as an optimization integration engine meticulously designed to facilitate communication between optimization clients and solver backends, emphasizing the need for high-performance and stable transport layers. The architecture primarily supports Linear Programming (LP) and Quadratic Programming (QP) optimization problems, restricting its operational scope to these mathematical realms.

The OptArrow architecture thrives on the in-memory data transfer capabilities provided by Apache Arrow, promoting seamless cross-language compatibility which is a crucial requirement given the diverse environments from which OptArrow can be invoked, including Python, Julia, and MATLAB. The use of Apache Arrow enables smooth and efficient data interchange between languages, thereby reducing overheads, ensuring that optimization tasks are performed efficiently across different computational platforms.

1.2 Core System Logic and Functionality

At its core, OptArrow receives optimization problems primarily coded in Python or Julia. The architecture is structured to discern the language of origin and accordingly route the problem through the appropriate computational pathway. Such precise routing pathways are essential to ensure that the solver backend employed is inline with the problem's origin. Upon completion of the optimization task, the solver outputs the results in the language originally used by the coder, facilitating smooth integration and reducing the need for any additional data handling by the user.

This routing mechanism involves a transformation of data representation from Python dictionaries to Arrow IPC (Inter-Process Communication) format followed by potential conversion to Julia structures, highlighting the efficiency and extensibility of the system architecture.

2. Stakeholder Needs & Feedback Mechanisms

2.1 Stakeholder Identification

OptArrow prioritizes a wide array of stakeholders including Software Engineers, Researchers, Data Scientists, and Optimization Specialists. These stakeholders are instrumental in augmenting the functionality of OptArrow as they bring forth diverse requirements and use cases that steer the ongoing evolution of the platform.

2.2 Feedback Collection Mechanisms

The process of collecting feedback from stakeholders is currently managed via the GitHub Issues mechanism. This channel allows stakeholders to submit their feedback, which is

invaluable for capturing real-time user experiences and requirements. However, the integration of this feedback into the formal requirements and design processes currently lacks an established traceability framework. While feedback is diligently recorded, the absence of a structured path linking stakeholder feedback directly to implemented decisions is a noted gap, denoting a forward-looking area for improvement.

3. Functional Requirements

3.1 Functional Requirement Specifications

OptArrow's core functional requirements are centered on its ability to accurately solve LP and QP optimization problems using trusted solver backends. The following encapsulates the system's key functional requirements:

- **FR-01:** The system shall accept LP and QP optimization problems as input.
- **FR-02:** The system shall interpret the language of origin (Python or Julia) and appropriately route the optimization problem to the corresponding solver backend.
- **FR-03:** The system shall utilize Apache Arrow to facilitate cross-language data exchange between Python and Julia environments efficiently.
- **FR-04:** The system shall output the optimization results in the same language as the input problem, preserving the original data structures wherever feasible.

3.2 Unique Identifiers for Each Requirement

Each functional requirement is uniquely identified to delineate its specific role within the broader system design. The identifiers FR-01 to FR-04 serve as a reference for mapping individual capabilities back to the documented architectural objectives.

4. Non-Functional Requirements

4.1 Performance Indicators

OptArrow's performance is defined by its ability to process optimization problems with minimal latency, maintaining both accuracy and computational efficiency. Measurable performance targets, however, are yet to be firmly established within the current documentation.

- **NFR-01:** The data transfer throughput via Apache Arrow shall support high-performance, low-latency communication between Python and Julia processes.
- **NFR-02:** The optimization solver shall operate within acceptable performance benchmarks to be established upon further development.

4.2 Compatibility Requirements

Given its function as a cross-language engine, OptArrow's compatibility requirements are integral to its operation. The adoption of Apache Arrow significantly contributes to achieving the desired levels of cross-language compatibility, ensuring OptArrow can seamlessly function within diverse development environments and infrastructures.

- **NFR-03:** The system shall maintain compatibility across Python, Julia, and MATLAB environments, allowing for straightforward invocation from each environment without necessitating amendment to the core system logic.

Each non-functional requirement, identified by unique identifiers (NFR-01 to NFR-03), serves to reinforce the system's operational integrity and adaptability within the specified operational contexts.

5. Requirements Traceability & Governance

5.1 Traceability Mechanisms

Despite the evident use of GitHub Issues for stakeholder feedback collection, there exists a confirmed gap in the formal traceability between the feedback collected and the requirements that subsequently guide system development. Future system enhancements must integrate a robust governance model designed to link each feedback/comment directly to implemented features/decisions.

The lack of this traceability presents an area ripe for development and improvement. Contemporary best practices recommend adopting comprehensive traceability matrices or utilizing automated tools to bridge this distance effectively.

5.2 Governance Models for Requirements Changes

Currently, a formal governance framework for effecting changes to OptArrow's architectural or functional requirements is yet to be documented. The absence of such governance formalities further underpins the criticality of developing structured, comprehensive governance protocols. Emphasis on the establishment of a documented change management process should be prioritized, encapsulating decision-making frameworks, accountability structures, and regular review cycles to ensure systematic evolution and maturation of system capabilities.

In summary, the OptArrow system specification document provides a robust outline of its existing functional and non-functional requirements, considering all verified gaps and areas for future innovation. Attention is focused on refining these processes further, guided by validated claims and industry-best practices to bolster OptArrow towards achieving greater optimization efficacy and stakeholder satisfaction.